Recommender systems

Lecturer: Assoc.Prof. Nguyễn Phương Thái

VNU University of Engineering and Technology
Slide: *from Assoc.Prof. Phan Xuân Hiếu, Updated: April 25, 2024*

What is a recommender system?

Definition from Nvidia

*A recommendation system is an **artificial intelligence** algorithm, usually associated with **machine learning**, that uses **big data** to suggest or recommend **additional products** to consumers. These can be based on various criteria, including **past purchases**, **search history**, **demographic information**, and other factors. Recommender systems are highly useful as they help users **discover products** and **services** they might otherwise have not found on their own.*

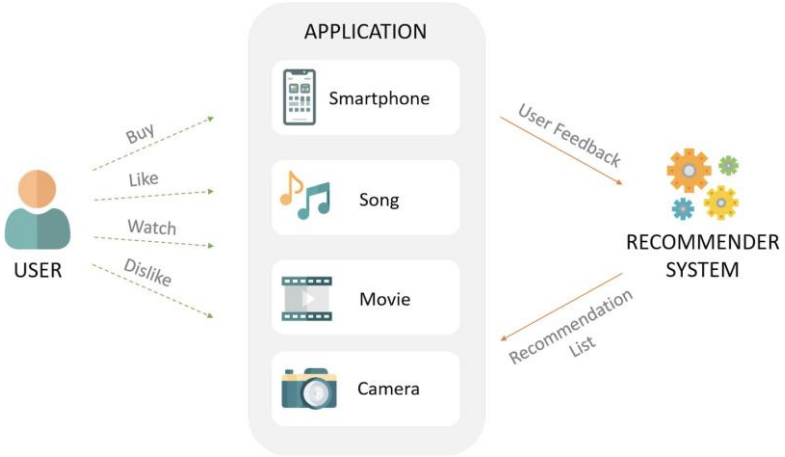
Definition from Wikipedia

*A recommender system, or a recommendation system, is a subclass of **information filtering** system that provide **suggestions for items** that are most **pertinent** to a particular user.*

Definition from ChatGPT

A recommender system is a type of information filtering system that predicts and recommends items to users based on their past behavior, preferences, and interests. It is commonly used in e-commerce, social media, online content platforms, and other applications where personalized recommendations can improve the user experience.

What is a recommender system? (cont'd)



[source: Internet]

Two primary approaches of recommender systems

1. Recommendation as a prediction problem
2. Recommendation as a ranking problem

Recommendation as a prediction problem

- Predicting the rating value for a user–item combination.
- Assuming that the *training* data is available, indicating user preferences for items.
- For m users and n items, the user ratings are an *incomplete* $m \times n$ matrix:
  - The *observed* (existing) values are used for training (e.g., a recommendation model).
  - The *unobserved* (missing) values will be predicted using the model.
- This problem is also referred to as the *matrix completion problem*.

Recommendation as a ranking problem

- Not necessary to predict the missing ratings to make recommendations.
- Rather:
  - Recommend the top–k items for a particular user; or
  - Determine the top–k users to target for a particular item.
- Determining top–k items is more common than determining top–k users.
- This problem is also referred to as the *top–k recommendation problem*.
- In this approach, the absolute values of the predicted ratings are not important.
- The ranking can be based on the predicted values in the first approach. However, it is easier and more natural to solve the ranking problem directly.

Goals of recommender systems

- **Business–centric goals**: the primary goals of a recommendation system are to
  - increase the user activities (view, watch, like, share, add to cart, etc.), user attention, user experience, and user engagement;
  - and finally, increase the sale volume, revenue and profit of the business service.
- **The operational and technical goals**:
  1. Relevance
  2. Novelty
  3. Serendipity
  4. Diversity
  5. Long–tail
  6. Fairness
  7. Other business and domain–specific goals

Goals of recommender systems (cont'd)

1. **Relevance**: recommend items that are relevant to the user:
   - Users are more likely to consume items that they find **interesting**; or
   - Or items that fit their **need** and **intents** (e.g., purchase intents).
2. **Novelty**: recommend items that the user has **not seen** in the past.
   - For example, popular movies of a preferred genre would rarely be novel to the user.
   - Repeated suggestion of popular items can also lead to reduction in sales diversity.
3. **Serendipity**: the recommended items are **truly surprising** to the user.
   - Rather than simply something they did not know about before.
   - This means something "*unknown unknown*" whereas novelty is "*unknown known*''.
   - Serendipity has the beneficial side effect of **increasing sales diversity** or **beginning a new trend** of interest in the user.
   - Serendipitous recommendations often tend to **irrelevant** items **at the beginning**.

Goals of recommender systems (cont'd)

4. **Diversity**: when all the recommended items are **very similar**, it increases the risk that the user **might not like any** of these items. Diversity help ensure that the user does not **get bored** by repeated recommendation of similar items.
5. **Long–tail**: recommender systems normally ignore unpopular or newly introduced items having only few ratings. Leveraging unpopular items at the long–tail of the distribution is important.
6. **Fairness**: this concept is related to groups of users based on race, gender, age, etc. For example, a job recommender system should have the fairness to suggest jobs to different races and genders.
7. **Other business and domain–specific goals**: this depends on the business applications. Some domains require explanation to the recommended items; Some need direct suggestions while some are indirect recommendation.

Some typical recommender systems

- GroupLens recommender system:
  - A pioneering prototype recommender system (recommendation of Usenet news), later extended to book and movie recommendation like BookLens and MovieLens.
  - GroupLens is now a research lab at The University of Minnesota, and has made pioneering contributions to collaborative filtering research.
  - Have released important datasets in this field like MovieLens with different sizes (https://grouplens.org/datasets/movielens).
- Amazon.com recommender system:
  - One of the pioneers in recommender systems, especially in e–commerce. Starting with book recommendation, and later expanded to virtually all forms of products.
  - Amazon recommendations are based on both *implicit* ratings (like browsing and buying behaviors) as well as *explicit* ratings (5–point scale, from 1–star to 5–star).
  - Recommendations are based on both (item–based) collaborative filtering and frequent patterns (e.g., *frequently bought together*, *bought this also bought that*).

Some typical recommender systems (cont'd)

- Netflix movie recommender system:
  - Recommendation is built based on 5–point scale ratings.
  - Providing meaningful *explanations* for the recommended items to users.
- Netflix Prize contest: predicting ratings of particular user–item combinations.
  - Training data set: 100,480,507 ratings (480,189 users and 17,770 movies), including
    - Probe set: 1,408,395 ratings.
  - Qualifying data set: 2,817,131 ratings (hidden), consisting of:
    - Quiz set: 1,408,342 ratings; used to calculate leaderboard scores.
    - Test set: 1,408,789 ratings; used to determine winners.
  - Data contain no information about users and movies except their IDs and ratings.
  - Submitted predictions are scored against the true grades using RMSE.
  - The ranking is based on the improvement over the Netflix's own algorithm Cinematch.
  - The grand prize of US$1M given to the BellKor's Pragmatic Chaos team in 2009.

Some typical recommender systems (cont'd)

- Google news personalization system:
  - Recommend news to users based on their history of clicks.
  - News articles are treated as items. User clicking on a news article is a positive rating.
  - Recommendations are based on *collaborative filtering* with these *implicit* ratings.
- Facebook friend recommendations:
  - Recommend potential friends to users in order to increase the social connections.
  - This has slightly different goals than a product recommendation.
  - This is also referred to as *link prediction* in the field of *social network analysis*.
  - Recommendations are based on *structural relationships* rather than rating data.

Some typical recommender systems (cont'd)

System	Domain
MovieLens	Movies
Amazon.com	E-commerce (books and other products)
Netflix	DVDs and Streaming videos
Google News	Online news
Google Search	Advertising
Facebook	Friends and Advertising
Pinterest	Photos
Tiktok	Short video clips (various domains)
Youtube	Videos (various domains)
Pandora	Music
Spotify	Music
Tripadvisor	Travel products
IMDb	Movies

Major approaches to building recommender systems

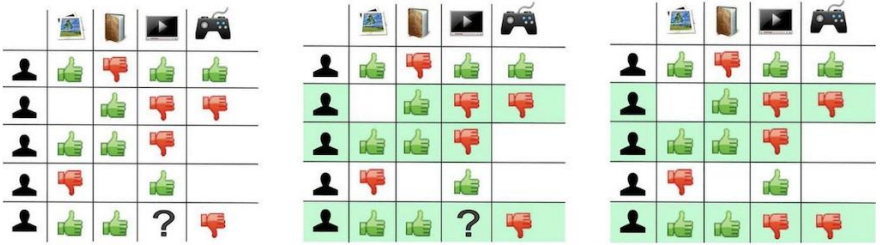
- Collaborative filtering recommender systems
- Content–based recommender systems
- Knowledge–based recommender systems
- Hybrid and ensemble–based recommender systems

Collaborative filtering recommender systems

- **Collaborative filtering** (**CF**) is an approach of making predictions about the interests of a user by collecting preferences or taste information from many users.

  The underlying assumption of CF is that if a person u has the same opinion as a person v on an issue, u is more likely to have v's opinion on another issue than that of a randomly chosen person.

- Collaborative filtering models use the collaborative power of the **ratings** provided by multiple users to make recommendations.

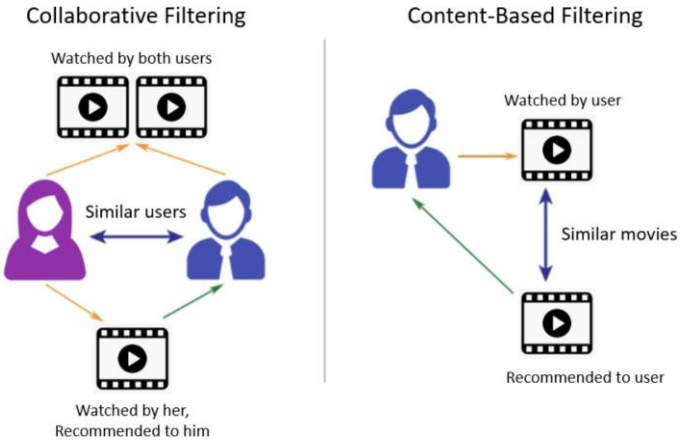


[source: Internet]

Content–based recommender systems

- The main idea of content–based recommendation is to suggest items **similar** to the items that the user **interacted** with in **the past**. *Interacted* means
  - User actions: browsing, reading, watching, add–to–cart, buying, sharing, etc.
  - Or user explicit ratings to the items.
- The recommendations for a particular user are based on the following information:
  - The **descriptive attributes** and the **content** of items.
  - The **basic info** of the user, e.g., demographic info (gender, age, etc.) if existing.
  - The **behaviors** of the user in the past, i.e., the **interaction history** of the user.
- There is no collaboration, i.e., recommendations for different users are independent.
- Content–based recommendation has its own pros and cons in comparison with CF.

Content–based versus collaborative filtering



[source: Internet]

Knowledge–based recommender systems

- Knowledge–based recommender systems are particularly useful for items that are **not purchased often**, i.e., ratings may **not be sufficient** for recommendation.
  - Examples: *real estate*, *automobiles*, *financial services*, or *expensive luxury goods*.
  - The nature of consumer preferences may evolve over time with such items.
- The recommendations are based on the basis of **similarities** between **customer requirements** and item descriptions, or the use of **customer constraints**.
  - Knowledge-based systems allow the users to **explicitly specify what they want**.
  - The explicit specification results in greater control of the recommendation process.

Approach	Conceptual Goal	Input
Collaborative	Give me recommendations based on a collaborative approach that leverages the ratings and actions of my peers/myself.	User ratings + community ratings
Content-based	Give me recommendations based on the content (attributes) I have favored in my past ratings and actions.	User ratings + item attributes
Knowledge-based	Give me recommendations based on my explicit specification of the kind of content (attributes) I want.	User specification + item attributes + domain knowledge

Hybrid and ensemble–based recommender systems

- In many cases, a wide variety of inputs is available:
  - User demographic information: gender, age, location, prior preferences, etc.
  - Item information: genre, description texts, tags, etc.
  - User ratings: implicit or explicit ratings, etc.
- In such cases, a **hybrid approach** is appropriate to leverage all types of data.
  - Combining CF, content–based, knowledge–based, etc.
  - Taking advantages of the strong points of those approaches.
- Several scenarios to build hybrid recommender systems:
  - Adding content information to a CF system.
  - Adding CF information to a content–based system.
  - Adding knowledge to a CF or content–based system.
  - Etc.

Context and social factors in recommender systems

- **Status or condition information**:
  - Context–ware: recommendations are also based on additional information (if existing).
  - For example: user is single, just married, being pregnant, etc.
- **Temporal information**:
  - Integrating temporal info: time in day, seasons, holidays, product life cycle, etc.
- **Location information**:
  - Recommendations are also based on the current location of users or items (if available).
  - Recommending nearest hotels, restaurants, etc.
- **Social information**:
  - Recommendations are also based on social tags, cues, viral info, social trending, etc.

Challenges and advanced topics in recommender systems

- The sparse data problem
- The cold–start problem in recommender systems
- The scalability in recommender systems Attack–
- resistant recommender systems
- Group recommender systems Multi–
- criteria recommender systems
- Privacy in recommender systems

The sparse data problem

- The user interaction and rating are limited, the rating matrices may be very sparse.
- Causing difficulty in computing similarity between users and between items.
  - Two very similar items may not be rated by same users.
  - Two users with similar taste (preferences) may not rate the same items.
- Sparse data will lead to a low coverage (e.g., long–tail items).
- Solutions:
  - Using matrix factorization, topic analysis, dimensionality reduction, embeddings, etc.
  - Graph–based methods.
  - Combining CF and content–based (content–boosted).
  - Etc.

The cold–start problem in recommender systems

- The **new user** problem:
  - Have not rated any items.
  - No history information (interactions, behaviors, etc.)
- The **new item** problem:
  - Have not been rated by any users.
  - Even have not been seen or bought any time.
- Solutions:
  - Recommending *hot* or *popular* items, or random recommendations at the beginning.
  - The new item problem: computing the similarity using item informations.
  - The new user problem: computing the similarity using demographic info (if available).
  - Combining CF and content–based in different ways.

The scalability in recommender systems

- The rating matrices are normally very large (millions × millions).
- Computation on very large matrices is very time–consuming:
  - Computing the similarity between users or between items.
  - Finding k nearest neighbors (k most similar users).
  - Etc.
- Recommendations are normally real–time or near real–time.
- Solutions:
  - Dimensionality reduction using PCA, SVD, etc.
  - Item–based CF is normally faster and more stable than user–based CF.
  - Building model–based systems (usually large training time).

Attack–resistant recommender systems

- This normally happens in a platform having many different competitors.
- Example: on an e–commerce market or platform: some sellers would submit
  - **inflated ratings** of their own products to manipulate the recommendation ranking.
  - and **biased** or **negative reviews** to products of their competitors.
- Numerous sophisticated strategies developed for attacking recommender systems.

Group recommender systems

- Recommendation to a group rather than a single user.
- Examples:
  - A family watching movies or television.
  - A group of friends going to a restaurant.
  - Travel recommendation to a family.
- Recommendation for a group is more complicated than for a single user.
- Simple combination of the taste or preferences of individuals is not really effective.

Multi–criteria recommender systems

- Items in several domains have different aspects.
- Examples:
  - Hotel: room, view, breakfast, wifi, reception, close to the down town, etc.
  - Movie: screenplay, acting, graphics, music, etc.
  - Smartphone: brand, display, battery, stability, software and apps, etc.
- Two users having the same rating score may not have similar taste or preferences.
- Multi–criteria systems help to rank items when considering multiple aspects.

Privacy in recommender systems

- Recommender systems are based heavily on feedback from the users, which might be implicit or explicit.
- This feedback contains significant information about the interests of the user. For examples:
  - Personal perferences,
  - Sexual orientations,
  - Political opinions,
  - Etc.
- Such information can be highly sensitive, which leads to privacy concerns.
- Privacy preserving in recommendation is really important.

Types of collaborative filtering

- **Memory–based CF methods**: also known as **neighborhood–based CF**. The ratings of user–item combinations are predicted on the basis of their neighborhoods. These neighborhoods can be defined in one of the two following ways:
  - **User–based CF**: recommendations are based on the similarity between users.
  - **Item–based CF**: recommendations are based on the similarity between items.

  – Memory–based methods are easy to implement.

  – The resulting recommendations are often easy to explain.

  – However, they do not work very well with *sparse* rating matrices.

- **Model–based CF methods**: machine learning and data mining methods are used in the context of *predictive* models. Typical model–based methods: decision trees, Bayesian methods, or latent factor models.

  – Models are parameterized and optimized; more complicated to implement.

  – Handling sparse rating matrices better.

Types of user ratings



source: from book [8]

- Unary rating: $\{\texttt{like}\}$ (e.g., Facebook like button; no way to express a dislike)
- Binary rating: $\{\texttt{like}, \texttt{dislike}\}$
- 5–point rating scale (discrete): $\{1, 2, 3, 4, 5\}$ or $\{-2, -1, 0, 1, 2\}$.
  – Some other options are 7–point or 10–point scale.
- Continuous rating values: e.g., any real value in $[-10, 10]$.

Explicit ratings versus implicit ratings

- **Explicit ratings**: users rate items explicitly using rating scale like unary, binary, 5–point scale, etc.
- **Implicit ratings**: users do not really rate items explicitly.
  - The rating values are inferred from the user actions like *browsing*, *reading*, *watching*, *add–to–cart*, *buying*, *sharing*, etc.
  - The rating values are based on types of actions, e.g., buying an item has a higher weight than just browsing it.

Rating matrices

- Let I be the set of n items in a system, e.g., n products, n movies, etc.
- Let U be the set of m users of the system.
- Let $R = \{r_{ui}\}_{m \times n}$ be the user–item rating matrix.
  - Where $r_{ui} \in \mathsf{R}$ is the rating value of the item i given by the user u.
  - Normally, only a small fraction of ratings are *specified* (or *observed*).
  - Most of the ratings are *missing* (i.e., *unspecified* or *unobserved*).
  - Thus, R is normally very *sparse*.
- The rating matrix is also called the user–item matrix or the utility matrix.
- Two types of rating matrix:
  - Explicit rating matrix: the rating values are given explicitly by users.
  - Implicit rating matrix: the rating values are inferred from the user actions.

Examples of user–item rating matrices [8]

	GLADITOR	GODFTHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U_1	1			5		2
U_2		5			4	
U_3	5	3		1		
U_4			3			4
U_5				3	5	
U_6	5		4			

(a) Ordered ratings

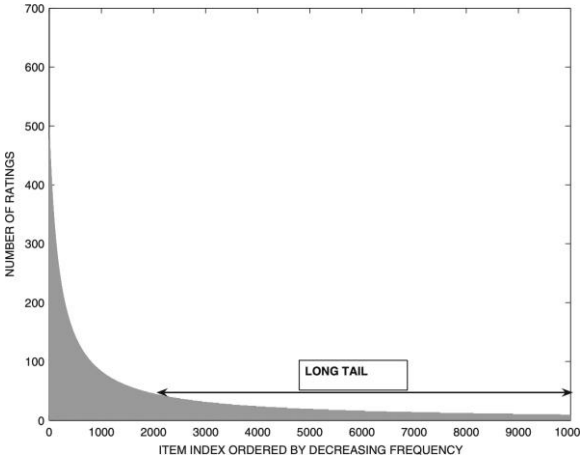
	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U_1	1			1		1
U_2		1			1	
U_3	1	1		1		
U_4			1			1
U_5				1	1	
U_6	1		1			

(b) Unary ratings

The long–tail of rating frequencies

- Only a small fraction of the items are rated frequently. That are *popular* items.
- A majority of items are rated rarely, leading to a highly skewed rating distribution.
  - High–frequency items tend to be relatively competitive items with little profit.
  - Lower frequency items normally have larger profit margins.
  - Advantageous to the merchant to recommend lower frequency items.
  - Amazon.com analysis: make most of their profit by selling items in the long tail.
- In fact, many recommendation algorithms:
  - Tend to suggest popular items rather than infrequent items.
  - Having negative impact on diversity, and users may often become bored by receiving the same set of recommendations of popular items.
- Need to consider the long–tail problem when designing recommendation algorithms.

The long–tail of rating frequencies (cont'd)



source: from the textbook [8]

Neighborhood–based collaborative filtering

1. Predicting ratings with neighborhood–based methods
2. Clustering and neighborhood–based methods
3. Dimensionality reduction and neighborhood methods
4. Graph models for neighborhood–based methods

1. **Predicting ratings with neighborhood–based methods**
2. Clustering and neighborhood–based methods
3. Dimensionality reduction and neighborhood methods
4. Graph models for neighborhood–based methods

Predicting ratings with neighborhood–based methods

- The idea in neighborhood–based methods is to make recommendations based on:
  - the user–user similarity, or
  - the item–item similarity computed from the rating matrix.
- **Neighborhood** implies determining either similar users or similar items.
- Two basic principles used in neighborhood–based models:
  - **User–based**: Similar users have similar ratings on the same item. Therefore, if Alice and Bob have rated movies in a similar way in the past, then one can use Alice's observed rating on *Terminator* to predict Bob's unobserved rating on this movie.
  - **Item–based**: Similar items are rated in a similar way by the same user. Therefore, Bob's ratings on similar science fiction movies like *Alien* and *Predator* can be used to predict his rating on *Terminator*.

User–based neighborhood methods

- Let $u, v \in U$ be two users, i.e., corresponding to two rows in the rating matrix.
- Let I_u and I_v denote the sets of item indices (columns) for which ratings have been specified by the user u and the user v, respectively.
- Then, $I_u \cap I_v$ is the set of item indices that are both rated by u and v.
- Example:
  - $I_u = \{1, 3, 5\}$ means u only rated the first, third, and fifth items.
  - $I_v = \{1, 2, 3, 4\}$ means v only rated the first, second, third, and fourth items.
  - $I_u \cap I_v = \{1, 3\}$ means the items 1 and 3 were commonly rated by both u and v.
- It is quite common that $I_u \cap I_v = \emptyset$because rating matrices are generally sparse.
- How to compute the **similarity** (in terms of taste or preferences) between u and v?

Computing user–user similarity using cosine

- The cosine similarity between user u and user v is defined as follows:

$$\begin{aligned} \text{Sim}(u, v) &= \text{Cosine}(u, v) \\ &= \frac{\sum_{k \in I_u \cap I_v} r_{uk} \cdot r_{vk}}{\sqrt{\sum_{k \in I_u \cap I_v} r_{uk}^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} r_{vk}^2}} \end{aligned} \tag{1}$$

  where r_{uk} and r_{vk} are the rating values of user u and user v on item k, respectively.

- What if different users may have **different scales** of ratings? For example, one user might be biased toward **liking** most items, whereas another user might be biased toward **not liking** most of the items.
- Pearson correlation is one way to deal with different scales of ratings.

Computing user–user similarity using Pearson correlation

The Pearson correlation coefficient between user u and user v is defined as follows:

$$\begin{aligned}\text{Sim}(u,v) &= \text{Pearson}(u,v) \\ &= \frac{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u) \cdot (r_{vk} - \mu_v)}{\sqrt{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u)^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} (r_{vk} - \mu_v)^2}}\end{aligned} \tag{2}$$

where μ_u and μ_v the mean values of ratings of u and v, respectively.

r_{uk} and r_{vk} are the rating values of user u and user v on item k, respectively.

Computing user–user similarity using Pearson correlation (cont'd)

- There are two ways to compute μ_u and μ_v:

$$\mu_u = \frac{\sum_{k \in I_u \cap I_v} r_{uk}}{|I_u \cap I_v|} \quad \text{and} \quad \mu_v = \frac{\sum_{k \in I_u \cap I_v} r_{vk}}{|I_u \cap I_v|} \tag{3}$$

or:

$$\mu_u = \frac{\sum_{k \in I_u} r_{uk}}{|I_u|} \quad \text{and} \quad \mu_v = \frac{\sum_{k \in I_v} r_{vk}}{|I_v|} \tag{4}$$

- It is hard to argue that which way is better.
- In extreme cases, if $|I_u \cap I_v| = 1$ (only one common item), the first way is not usable. Even if $|I_u \cap I_v| = 2$ or 3 and those rating values of one user (u or v) are identical, the similarity will also be zero.
- The second way is more efficient in computation (computed once for each user). However, many implementations compute μ_u and μ_v in pairwise fashion.

Prediction function: predicting user ratings using weighted average

- Let u be the **active** or **target** user, i.e., whose ratings will be predicted.
- Let $\hat{r}_{uj}$ be the (unobserved) rating value of u on item j that we need to predict.
- The prediction scenario:
  - Finding top k most similar users $U_u(j)$ of the user u who already rated the item j using Person or cosine similarity. $U_u(j)$ is also called a **peer group** of u regarding item j.
  - Because different users may provide ratings on different scales. One might rate all items highly, whereas another might rate all items negatively. The raw ratings should be normalized using mean–centered in row–wise fashion:

$$s_{vj} = r_{vj} - \mu_v \quad \forall v \in \{1 \dots m\} \tag{5}$$

  - Computing the weighted average prediction for $\hat{r}_{uj}$ as follows:

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in U_u(j)} Sim(u,v) \cdot s_{vj}}{\sum_{v \in U_u(j)} |Sim(u,v)|} = \mu_u + \frac{\sum_{v \in U_u(j)} Sim(u,v) \cdot (r_{vj} - \mu_v)}{\sum_{v \in U_u(j)} |Sim(u,v)|} \tag{6}$$

Example of predicting user ratings using weighted average

Item-Id ⇒ / User-Id ⇓	1	2	3	4	5	6	Mean Rating	Cosine$(i,3)$ (user-user)	Pearson$(i,3)$ (user-user)
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

source: from the textbook [8]

- The table shows the ratings of five user $\{1 \dots 5\}$ on six items $\{1 \dots 6\}$.
- Each rating is drawn from the range $\{1 \dots 7\}$.
- Consider the current or target user index is $u = 3$.
- User–user similarity between the user u and other users using Cosine and Pearson.

Example of predicting user ratings using weighted average (cont'd)

- For example, computing the Cosine similarity between user 1 and u $(= 3)$:

$$\begin{aligned}\text{Cosine}(1,3) &= \frac{6 \cdot 3 + 7 \cdot 3 + 4 \cdot 1 + 5 \cdot 1}{\sqrt{6^2 + 7^2 + 4^2 + 5^2} \cdot \sqrt{3^2 + 3^2 + 1^2 + 1^2}} \\ &= 0.956\end{aligned}$$

- Computing the Pearson similarity:

$$\begin{aligned}&\text{Pearson}(1,3) \\ &= \frac{(6 - 5.5) \cdot (3 - 2) + (7 - 5.5) \cdot (3 - 2) + (4 - 5.5) \cdot (1 - 2) + (5 - 5.5) \cdot (1 - 2)}{\sqrt{1.5^2 + 1.5^2 + (-1.5)^2 + (-0.5)^2} \cdot \sqrt{1^2 + 1^2 + (-1)^2 + (-1)^2}} \\ &= 0.894\end{aligned}$$

Example of predicting user ratings using weighted average (cont'd)

- Let $k = 2$. The top–2 closest users of 3 are 1 and 2 according to both measures.
- Using the Pearson–weighted average of the *raw* ratings of users 1 and 2 to predict the missing rating values $\hat{r}_{31}$ and $\hat{r}_{36}$ as follows:

$$\hat{r}_{31} = \frac{7 \cdot 0.894 + 6 \cdot 0.939}{0.894 + 0.939} \approx 6.49$$

$$\hat{r}_{36} = \frac{4 \cdot 0.894 + 4 \cdot 0.939}{0.894 + 0.939} = 4.0$$

- The corresponding predictions with *mean–centered* ratings:

$$\hat{r}_{31} = 2 + \frac{1.5 \cdot 0.894 + 1.2 \cdot 0.939}{0.894 + 0.939} \approx 3.35$$

$$\hat{r}_{36} = 2 + \frac{-1.5 \cdot 0.894 - 0.8 \cdot 0.939}{0.894 + 0.939} \approx 0.86$$

Discounted similarity between users

- The **reliability** of the similarity function $\text{Sim}(u, v)$ is often affected by the number of common ratings $|I_u \cap I_v|$ between users u and v.
- When the number of common ratings between two users is **too small**, need to **de–emphasize** the importance of that user pair.
- Let β be a minimum threshold for the number of common ratings, the discounted similarity is defined as follows:

$$\text{DiscountedSim}(u, v) = \text{Sim}(u, v) \cdot \frac{\min\{|I_u \cap I_v|, \beta\}}{\beta} \tag{7}$$

- The discounted similarity is used both for the process of determining the peer group (i.e., the top–k closest users) and for computing the prediction.

Variants of the prediction function

- Use z–score normalization instead of mean–centering.
- The standard deviation for user v is defined as follows:

$$\sigma_v = \sqrt{\frac{\sum_{j \in I_v} (r_{vj} - \mu_v)^2}{|I_v| - 1}} \qquad \forall v \in \{1 \dots m\} \tag{8}$$

- Then, the normalized rating is computed as follows:

$$z_{vj} = \frac{r_{vj} - \mu_v}{\sigma_v} \tag{9}$$

- Then, the prediction function for the rating of user u on item j is:

$$\hat{r}_{uj} = \mu_u + \sigma_u \cdot \frac{\sum_{v \in U_u(j)} Sim(u, v) \cdot z_{vj}}{\sum_{v \in U_u(j)} |Sim(u, v)|} \tag{10}$$

Finding top–k closest users (i.e., filtering the peer group)

- Amplifying the similarity between users:

$$\text{Sim}(u, v) = \text{Pearson}(u, v)^{\alpha} \tag{11}$$

  Choosing $\alpha > 1$ if we want to amplify the importance of the similarity.

- We should remove *weakly* and *negatively* correlated users with the active user u from the top–k.
  - Weakly correlated users might add to the error in the prediction.
  - Negatively correlated ratings often do not have as much predictive value in terms of potential inversion of the ratings (although the prediction function technically allows the use of weak or negative ratings).

Impact of the long tail

- Some items may be **very popular** and they may repeatedly occur as **commonly rated** items by different users.
- Such ratings can sometimes worsen the quality of the recommendations because they tend to be **less discriminative** across different users.
- Need to **de–emphasize** those items. The idea is similar to **inverse document frequency** (idf) in tf–idf weighting. Each item j has a weight w_j:

$$w_j = \log\left(\frac{m}{m_j}\right) \quad \forall j \in \{1 \dots n\} \tag{12}$$

- Modifying the similarity, e.g., the Pearson correlation, as follows:

$$\text{Pearson}(u, v) = \frac{\sum_{k \in I_u \cap I_v} w_k \cdot (r_{uk} - \mu_u) \cdot (r_{vk} - \mu_v)}{\sqrt{\sum_{k \in I_u \cap I_v} w_k \cdot (r_{uk} - \mu_u)^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} w_k \cdot (r_{vk} - \mu_v)^2}} \tag{13}$$

Item–based neighborhood methods

- **Peer groups** are constructed in terms of *items* rather than *users*.
  - Similarities need to be computed between items (or columns in the ratings matrix).
  - Before computing the similarities between columns, each row is mean–centered.
- Let i be a user, U_i be the indices of the set of users who have specified ratings for item i. For example, if the first, third, and fourth users have rated item i, then $U_i = \{1, 3, 4\}$. $U_i \cap U_j$ is the set of user indices who rated both i and j.
- To compute the similarity between items i and j, the **adjusted cosine** is defined as:

$$\begin{aligned} \text{AdjustedCosine}(i,j) &= \frac{\sum_{v \in U_i \cap U_j} s_{vi} \cdot s_{vj}}{\sqrt{\sum_{v \in U_i \cap U_j} s_{vi}^2} \cdot \sqrt{\sum_{v \in U_i \cap U_j} s_{vj}^2}} \\ &= \frac{\sum_{v \in U_i \cap U_j} (r_{vi} - \mu_v) \cdot (r_{vj} - \mu_v)}{\sqrt{\sum_{v \in U_i \cap U_j} (r_{vi} - \mu_v)^2} \cdot \sqrt{\sum_{v \in U_i \cap U_j} (r_{vj} - \mu_v)^2}} \end{aligned} \tag{14}$$

Item–item based prediction function

- Let u be the active or target user. And let t be the item where the rating value r_{ut} is missing and we need to predict $\hat{r}_{ut}$.
- The basic idea is to leverage the user's **own** ratings on similar items to make prediction.
- The prediction scenario:
  - Determine top–k most similar *items* to item t based on adjusted cosine, denoted as I_t.
  - Let $I_t(u)$ be the set items in I_t that u has already rated.
  - The predicted rating $\hat{r}_{ut}$ of user u for the item t is computed based on *weighted* average as follows:

$$\hat{r}_{ut} = \frac{\sum_{j \in I_t(u)} \text{AdjustedCosine}(j,t) \cdot r_{uj}}{\sum_{j \in I_t(u)} |\text{AdjustedCosine}(j,t)|} \tag{15}$$

Row–normalized rating matrix for item–item similarity computation [8]

Item-Id ⇒ / User-Id ⇓	1	2	3	4	5	6	Mean Rating	Cosine$(i,3)$ (user-user)	Pearson$(i,3)$ (user-user)
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

Item-Id ⇒ / User-Id ⇓	1	2	3	4	5	6
1	1.5	0.5	1.5	-1.5	-0.5	-1.5
2	1.2	2.2	?	-0.8	-1.8	-0.8
3	?	1	1	-1	-1	?
4	-1.5	-0.5	-0.5	0.5	0.5	1.5
5	-1	?	-1	0	1	1
Cosine$(1,j)$ (item-item)	1	0.735	0.912	-0.848	-0.813	-0.990
Cosine$(6,j)$ (item-item)	-0.990	-0.622	-0.912	0.829	0.730	1

Example of item–item based prediction for rating values

- Predict the missing ratings of the user 3 for items 1 and 6, i.e., estimating $\hat{r}_{31}$, $\hat{r}_{36}$.
- Similarity values between items are computed using AdjustedCosine, e.g.:

$$\text{AdjustedCosine}(1,3) = \frac{1.5 \cdot 1.5 + (-1.5) \cdot (-0.5) + (-1) \cdot (-1)}{\sqrt{1.5^2 + (-1.5)^2 + (-1)^2} \cdot \sqrt{1.5^2 + (-0.5)^2 + (-1)^2}} = 0.912$$

- Items 2 and 3 are most similar to item 1:

$$\hat{r}_{31} = \frac{3 \cdot 0.735 + 3 \cdot 0.912}{0.735 + 0.912} = 3.0 \quad \text{(vs. } 3.35 \text{ of user–based)}$$

- Items 4 and 5 are most similar to item 6:

$$\hat{r}_{36} = \frac{1 \cdot 0.829 + 1 \cdot 0.730}{0.829 + 0.730} = 1.0 \quad \text{(vs. } 0.86 \text{ of user–based)}$$

Efficient implementation and computational complexity

- Store **intermediate computations** for efficient ranking of recommendations.
- The recommendation process is divided into **offline** phase and **online** phase:
  - **Offline**: (**a**) user–user (item–item) similarity values and peer groups (of user/item) are computed; (**b**) for each user (or item), the relevant peer group is prestored.
  - **Online**: those prestored similarity values and peer groups are used to make prediction.
- Let n' ($\ll n$) be the maximum number of specified ratings of a user (row); and m' ($\ll m$) be the maximum number of specified ratings of an item (column).
  - n' is the maximum running time for computing the similarity between a pair of users.
  - m' is the maximum running time for computing the similarity between a pair of items.
  - Identifying the peer group of an active/target user (user–based): $O(m \cdot n')$.
  - Identifying the peer group of an active/target item (item–based): $O(n \cdot m')$.
  - Computing the peer groups of all users is $O(m^2 \cdot n')$.
  - Computing the peer groups of all items is $O(n^2 \cdot m')$.

Efficient implementation and computational complexity (cont'd)

- Space complexity to store all pairs of similarities between pairs of users: $O(m^2)$.
- Space complexity to store all pairs of similarities between pairs of items: $O(n^2)$.
- Normally $m > n$, user–based methods need more space than item–based ones.
- **Online phase**:
  - Let k be the size of the user/item neighborhood used for prediction.
  - Time complexity to rank all items for a target user: $O(k \cdot n)$ (user– or item–based).
  - Time complexity to rank all users for a specific item: $O(k \cdot m)$.
- Time complexity for the offline phase is much larger than that of the online phase.
- In real implementation, various **pre–computing** and **caching** techniques should be used to accelerate both phase 1 and phase 2 (need near real–time recommendation).

Comparing user–based and item–based methods

- Item–based methods often provide **more relevant** recommendations.
  - Because user's **own** ratings are used **directly** to perform the recommendation. User–
  - based: ratings are extrapolated from other users (may have different interests).
- User–based methods may provide **more diverse** and **novel** recommendations.
  - Novelty, diversity, and serendipity mean more surprising, interesting, and discovery.
- Item–based methods can provide concrete reasons for recommendations, e.g.:
  - Because you watched "Secrets of the Wings", the recommendations are ...
- Item–based methods are **more stable** with changes to the ratings because
  - The number of users is generally much larger than the number of items: two users may have a very small number of mutually rated items while two items are likely to have a large number of users co–rating them. Thus, with user–based, adding a few ratings can change the similarity between users significantly.
  - New users are likely to be added more frequently in than new items. Thus, the similarity values between items are more stable than those between users. This also leads to less re–computation frequency for item–based methods.

Pros and cons of neighborhood–based methods

- **Strengths**:
  - Simplicity: easy to implement and debug.
  - An intuitive approach: easy to justify why a specific item is recommended.
  - Relatively stable with the addition of new items and users.
  - Incremental: easy to expand.
- **Weaknesses**:
  - Computation is expensive with large–scale settings, especially for the offline phase.
  - Likely low coverage due to the data sparsity.

1. Predicting ratings with neighborhood–based methods
2. **Clustering and neighborhood–based methods**
3. Dimensionality reduction and neighborhood methods
4. Graph models for neighborhood–based methods

Clustering and neighborhood–based methods

- Main problem of neighborhood–based: the **huge complexity** of the offline phase:
  - $O(m^2 \cdot n')$ is **impractical** if the number of users m is a few hundred million.
  - E.g.: $m = 10^8$ and $n' = 100$, $O(m^2 \cdot n') = O(10^{18})$.
- **Idea**: replacing the offline neighborhood computation with an **offline clustering**.
  - Problem: offline nearest–neighbor phase creates a **large** number of peer groups.
  - Solution: clustering process creates a **smaller** number of peer groups.
  - Clustering is much more efficient than the $O(m^2 \cdot n')$.
  - After clustering: top–k closest peers within the **same cluster** are used for prediction.
- Pros and cons of combining clustering with neighborhood–based:
  - Cluster size is **much smaller** than m (or n), finding top–k closest peers is **efficient**.
  - Some **loss** of accuracy due to **locality** (i.e., only searching peers within cluster).
  - Clusters are more **fine–grained**, the efficiency improves, but the accuracy is reduced.
  - In many cases, very **large gains** in efficiency only **pay a small** reduction in accuracy.

Clustering on the rating matrix

- Challenge: the rating matrix is **incomplete**.
  - Clustering methods need to be adapted to work with massively incomplete data sets.
- Clustering methods like *k*–**means** can be easily adapted to incomplete data.
  - Problem: the $m \times n$ rating matrix is incomplete.
  - Computation of the **mean** and the **distance values** becomes **undefined**.
  - Solution: compute the means using only the **observed values** within a cluster.
  - The distance values are computed using only the **subset of dimensions**, which are specified both for the data point and cluster representative.
  - **Manhattan** distance is **more appropriate** than the Euclidean or other distances.
- Clustering can be performed for both **users** (rows) and **items** (columns).

1. Predicting ratings with neighborhood–based methods
2. Clustering and neighborhood–based methods
3. **Dimensionality reduction and neighborhood methods**
4. Graph models for neighborhood–based methods

Dimensionality reduction and neighborhood methods

- **Dimensionality reduction** (**DR**) can improve both **quality** and **efficiency**.
  - DR provides a **dense low–dimensional** representation in terms of **latent factors**.
  - Computing similarity between dense low–dimensional vectors is easier and efficient.
  - Thus, able to find **more relevant** neighbors with **less time**.
  - There are **two ways** of applying DR for recommender systems.
- **First way**: DR is applied for **either** users (row–wise) or items (column–wise):
  - A reduced representation will either **compress** the item dimensionality or the user dimensionality into **latent factors**.
  - This reduced representation can be used to **alleviate** the **sparsity** problem for neighborhood–based models.
- **Second way**: DR is applied for **both** users and items **simultaneously**:
  - This belongs to the **model–based** CF approach. Will be discussed later.

Dimensionality reduction for user–based collaborative filtering

- Idea: **transform** the $m \times n$ rating matrix R into a **lower–dimensional space** using dimensionality reduction techniques.
  - The resulting matrix $R^{'}$ of size $m \times d$ where $d \ll n$.
  - Each n–dimensional rating vector of a user is transformed into a d–dimensional vector.
  - Unlike the their original vectors, d–dimensional vectors are **dense**.
  - Computing similarity between d–dimensional vectors are more accurate and efficient.
- Typical dimensionality reduction techniques are:
  - Principal component analysis (PCA),
  - Singular value decomposition (SVD),
  - Etc.

1. Predicting ratings with neighborhood–based methods
2. Clustering and neighborhood–based methods
3. Dimensionality reduction and neighborhood methods
4. **Graph models for neighborhood–based methods**

Graph models for neighborhood–based methods

- The sparsity of observed ratings causes a major problem in similarity computation.
- Graph–based is an approach to define similarity in neighborhood–based methods.
- The general idea is the use of **structural transitivity** on graphs to:
  - infer and estimate the relationships among users and/or items, and
  - ranking recommendations.
- Graphs can be constructed on the **users**, on the **items**, or on **both**.
- A **variety** of algorithms on graphs, e.g., **random walk** or **shortest path**, etc.

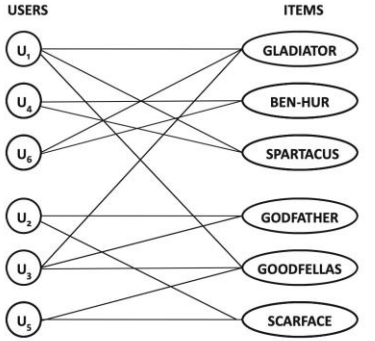
User–item graphs

- The user–item graph is an **undirected** and **bipartite** graph: $\mathbf{G} = (N_u \cup N_i, A)$.
  - N_u is the set of nodes representing users.
  - N_i is the set of nodes representing items.
  - All edges in the graph exist only between users and items.
  - An undirected edge exists in A between user i and item j iff user i has rated item j.
  - Therefore, $|A|$ = the number of observed entries in the rating matrix.
- **Advantage**: two users do not need to have rated many of the same items to be considered neighbors as long as many short paths exist between the two users.
  - Neighborhoods can be defined via **indirect connectivity** between nodes.
  - Of course, **direct** or **shorter** paths have **stronger** weights when computing similarity.
  - Different ways of defining neighborhoods on graphs to deal with sparse data problem.

User–item graphs (cont'd)

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U_1	1			5		2
U_2		5			4	
U_3	5	3		1		
U_4			3			4
U_5				3	5	
U_6	5		4			

(a) Ratings matrix



(b) User-item graphs of specified ratings

source: from the textbook [8]

Defining neighborhoods with random walks

- The neighborhood of a user is defined by the set of users that are **encountered frequently** in a **random walk** starting at that user.
- How can the expected frequency of such random walks be measured?
  - Using personalized PageRank or the SimRank methods
  - to determine the k most similar users to a given user for user-based CF, or
  - to determine the k most similar items to a given item by starting the random walk at a given item.
- Why is this approach more effective for sparse matrices?
  - When computing Pearson correlation: two users need to be connected **directly** to a set of common items.
  - In sparse user–item graphs, such direct connectivity may not exist for many nodes.
  - A random–walk method also considers **indirect connectivity** to overcome sparsity.

Defining neighborhoods with the Katz measure

- Based on the **weighted** number of walks between a pair of nodes.
- The weight of each walk is in $(0, 1)$, which is a decreasing function of its length.
- The weighted number of walks between a pair of nodes is the **Katz measure**.
- **Intuition**: if two users belong to the same neighborhood based on walk–based connectivity, then there is a propensity for a link to be formed between them.
- **Katz measure**: Let $n_{ij}^{(t)}$ be the number of walks of length t between nodes i and j. Then, for a user–defined parameter $\beta < 1$, the Katz measure between i and j is:

$$\text{Katz}(i, j) = \sum_{t=1}^{\infty} \beta^t \cdot n_{ij}^{(t)} \tag{26}$$

The value of β is a discount factor that de–emphasizes walks of longer lengths. For small enough values of β, the infinite summation of Equation (26) will converge.

Defining neighborhoods with the Katz measure (cont'd)

- Let K be the $m \times m$ matrix of Katz coefficients between pairs of users.
- If A is the **symmetric adjacency matrix** of an undirected network, then the pairwise Katz coefficient matrix K can be computed as follows:

$$K = \sum_{i=1}^{\infty} (\beta A)^i = (I - \beta A)^{-1} - I \tag{27}$$

- The value of β should always be selected to be smaller than the inverse of the largest eigenvalue of A to ensure convergence of the infinite summation.
- A can be replaced with the **weight matrix** of the graph. This is to weight the edges in the user–item graph with the corresponding **rating**.
- Should apply a **threshold** for the **maximum path length** for less computation.

Item–item graphs

- An item–item graph is called a *correlation graph*.
- An item–item graph is a **weighted** and **directed** network $G = (N, A)$:
  - Each node in N is an item.
  - Each edge (i, j) in A is a relationship between items i and j; and having a weight w_{ij}.
  - If i and j are rated by at least one common user, both edges (i, j) and (j, i) exist in G.
- Computing the weight w_{ij} of the edge (i, j):
  - Let U_i and U_j be the sets of users who have rated item i and item j, respectively.
  - Initialize $w_{ij} = |U_i \cap U_j|$. At this point, $w_{ij} = w_{ji}$, i.e., symmetric.
  - Weight normalization: the sum of weights of outgoing edges of a node is equal to 1.

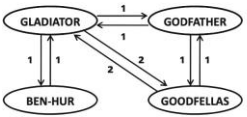
$$w'_{ij} = \frac{w_{ij}}{\sum_{(i,k)\in A} w_{ik}} \quad \text{and} \quad w'_{ji} = \frac{w_{ji}}{\sum_{(j,k)\in A} w_{jk}} \tag{28}$$

  - Now, the weights of edges in G correspond to *random–walk probabilities*.

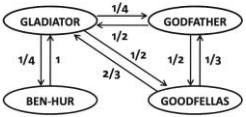
Example of item–item graph [8]

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS
U_1	1			5
U_2		5		
U_3	5	3		1
U_4			3	
U_5				3
U_6	5		4	

(a) Ratings matrix



(b) Unnormalized correlation graph



(c) Normalized correlation graph

Item–item graphs (cont'd)

- The weights on the *normalized* correlation graph are **not symmetric**.
- Weights can be defined by using cosine between the rating vectors of the two items.
- Random–walk methods can be used to determine the neighborhood of a given item.
- This graph is can be used in item–based collaborative filtering.

References

1 J. Han, M. Kamber, and J. Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Elsevier, 2012 [Book1].

2 C. Aggarwal. *Data Mining: The Textbook*. Springer, 2015 [Book2].

3 J. Leskovec, A. Rajaraman, and J. D. Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2014 [Book3].

4 M. J. Zaki and W. M. Jr. *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2013 [Book4].

5 D. Easley and J. Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010 [Book5].

6 J. VanderPlas. *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly, 2017 [Book6].

7 J. Grus. *Data Science from Scratch: First Principles with Python*. O'Reilly, 2015 [Book7].

8 C. Aggarwal. *Recommender Systems: The Textbook*. Springer, 2016 [Book8].

Summary

- Recommendation: prediction/ranking, goals, approaches, challenges, applications.
- Rating matrices, rating scales, explicit/implicit ratings, sparsity, long–tail, etc.
- Neighborhood–based CF:
  - Rating prediction: user–based, item–based, target/active user, peer groups, etc.
  - Pearson correlation, adjust cosine for user–user, item–item similarity computation.
  - User bias, rating normalization, mean–centering, etc.
  - Clustering–based neighborhood CF
  - Dimensionality reduction: SVD, Compact SVD, Truncated SVD, etc.
  - Graph models: structural transitivity, random walk, user–item and item–item graphs.